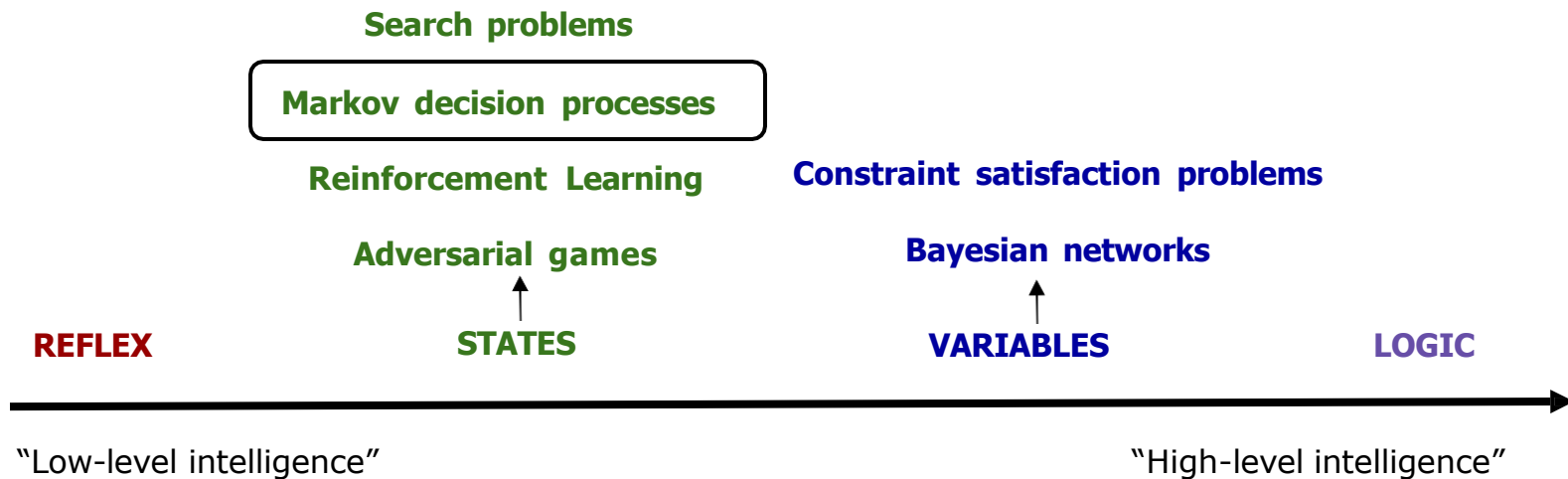


Topic 4: Markov Decision Process

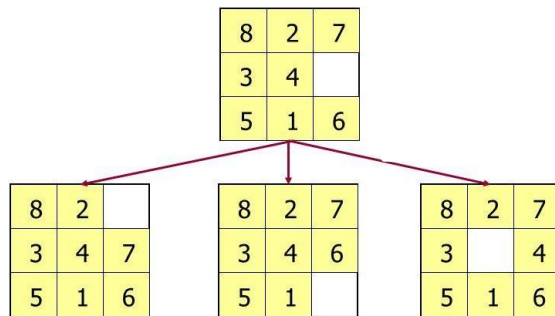


UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Course Plan:

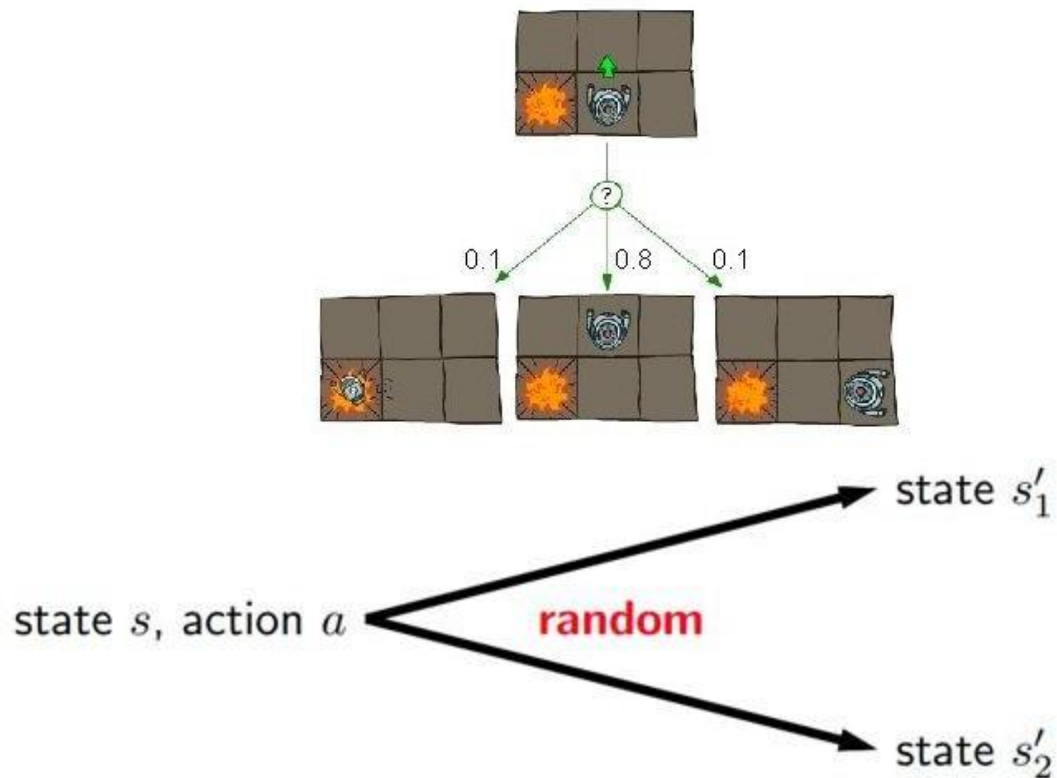


So far: search problems



state s , action a **deterministic**  state $\text{Succ}(s, a)$

Uncertainty in the real world



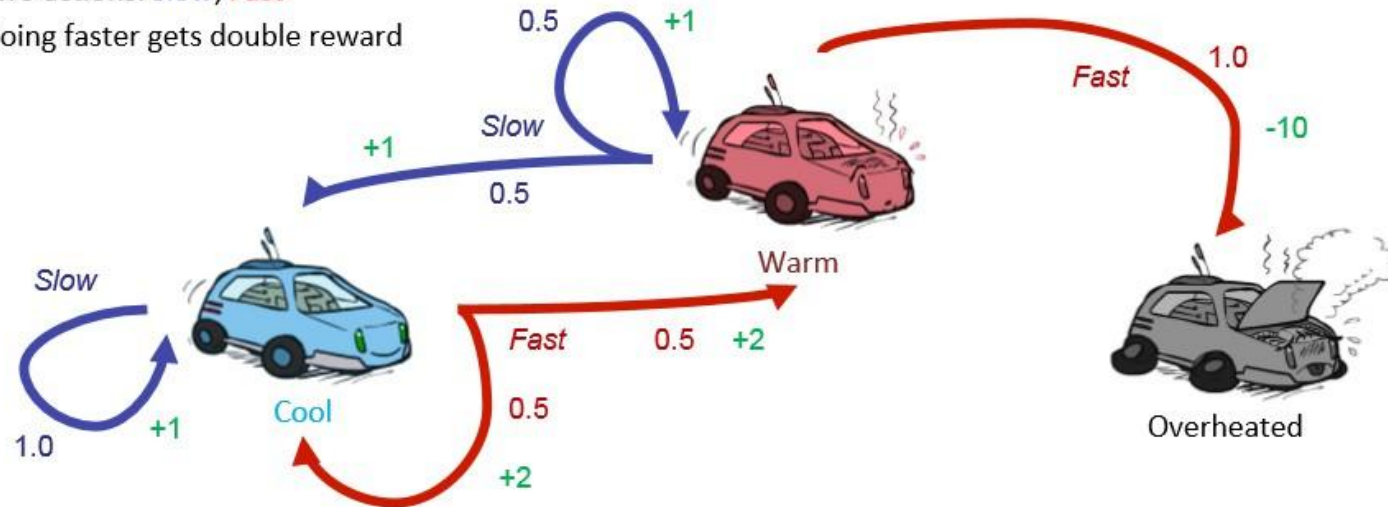
Example

A robot car wants to travel far, quickly

Three states: *Cool*, *Warm*, Overheated

Two actions: *Slow*, *Fast*

Going faster gets double reward



Applications

Robotics: decide where to move, but actuators can fail, hit unseen obstacles, etc



Resource allocation: decide what to produce, don't know the customer demand for various products



Finance: decide how much to invest in stock, don't know stock performance



Outline



- **Markov decision process**
- Policy evaluation
- Value iteration

Dice game



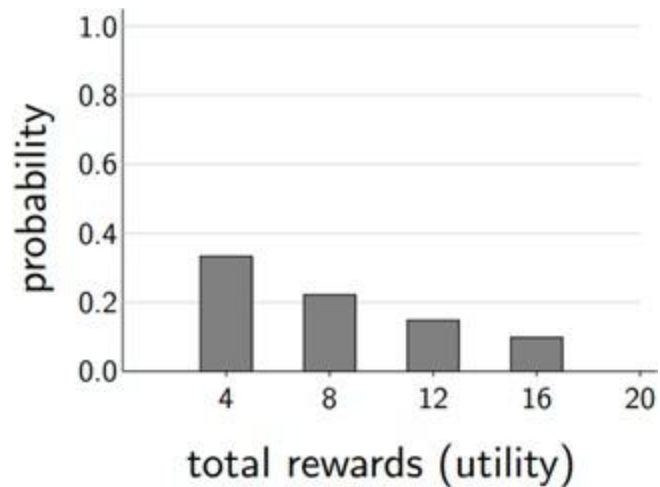
Example: dice game

For each round $r = 1, 2, \dots$

- You choose **stay** or **quit**.
- If **quit**, you get \$10 and we end the game.
- If **stay**, you get \$4 and then I roll a 6-sided dice.
 - If the dice results in 1 or 2, we end the game.
 - Otherwise, continue to the next round.

Rewards

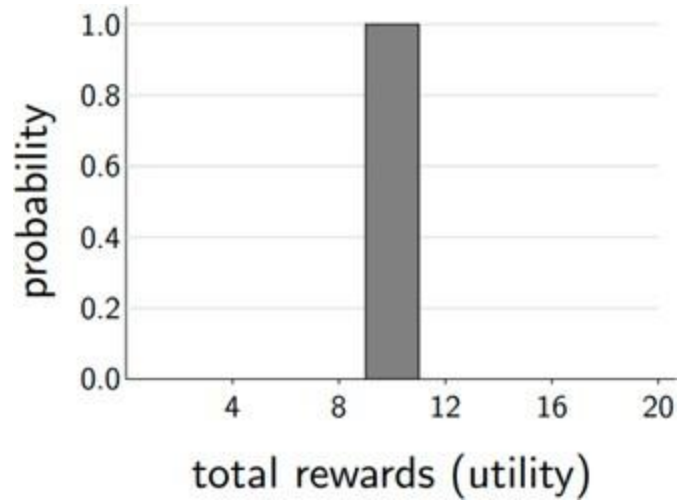
If follow policy "stay":



Expected utility: $\frac{1}{3}(4) + \frac{2}{3} \cdot \frac{1}{3}(8) + \frac{2}{3} \cdot \frac{2}{3} \cdot \frac{1}{3}(12) + \dots = 12$

Rewards

If follow policy "quit":



Expected utility: $1(10) = 10$

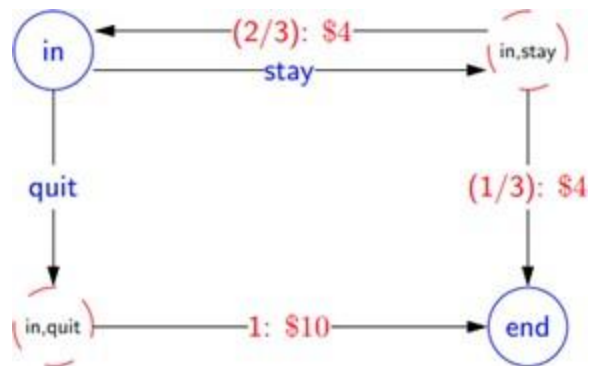
MDP for dice game



Example: dice game

For each round $r = 1, 2, \dots$

- You choose **stay** or **quit**.
- If **quit**, you get \$10 and we end the game.
- If **stay**, you get \$4 and then I roll a 6-sided dice.
 - If the dice results in 1 or 2, we end the game.
 - Otherwise, continue to the next round.



Markov decision process



Definition: Markov decision process

- States: the set of states
- $s_{\text{start}} \in \text{States}$: starting state
- $\text{Actions}(s)$: possible actions from state s
- $T(s' | s, a)$: probability of s' if take action a in state s
- $\text{Reward}(s, a, s')$: reward for the transition (s, a, s')
- $\text{IsEnd}(s)$: whether at end of game
- $0 \leq \gamma \leq 1$: discount factor (default: 1)

Search problems



Definition: search problem

- States: the set of states
 - $s_{\text{start}} \in \text{States}$: starting state
 - $\text{Actions}(s)$: possible actions from state s
 - $\text{Succ}(s, a)$: where we end up if take action a in state s
 - $\text{Cost}(s, a)$: cost for taking action a in state s
 - $\text{IsEnd}(s)$: whether at end
-
- $\text{Succ}(s, a) \Rightarrow T(s' \mid s, a)$
 - $\text{Cost}(s, a) \Rightarrow \text{Reward}(s, a, s')$

Transitions



Definition: transition probabilities

The **transition probabilities** $T(s' | s, a)$ specify the probability of ending up in state s' if taken action a in state s .



Example: transition probabilities

s	a	s'	$T(s' s, a)$
in	quit	end	1
in	stay	in	$2/3$
in	stay	end	$1/3$

Probabilities sum to one



Example: transition probabilities

s	a	s'	$T(s' s, a)$
in	quit	end	1
in	stay	in	$2/3$
in	stay	end	$1/3$

For each state s and action a : $\sum_{s' \in \text{States}} T(s' | s, a) = 1$

Successors: s' such that $T(s' | s, a) > 0$

Transportation example



Example: transportation

Street with blocks numbered 1 to n .

Walking from s to $s + 1$ takes 1 minute.

Taking a magic tram from s to $2s$ takes 2 minutes.

How to travel from 1 to n in the least time?

Tram fails with probability 0.5.

What is a solution?

Search problem: path (sequence of actions)

MDP:



Definition: policy

A policy π is a mapping from each state $s \in \text{States}$ to an action $a \in \text{Actions}(s)$.



Example: car racing

s	$\pi(s)$
cool	fast
warm	slow
...	...

Outline



- Markov decision process
- **Policy evaluation**
- Value iteration

Evaluating a policy



Definition: utility

Following a policy yields a **random path**.

The **utility** of a policy is the (discounted) sum of the rewards on the path (this is a random quantity).

Path	Utility
[in; stay, 4, end]	4
[in; stay, 4, in; stay, 4, in; stay, 4, end]	12
[in; stay, 4, in; stay, 4, end]	8
[in; stay, 4, in; stay, 4, in; stay, 4, in; stay, 4, end]	16
....	



Definition: value (expected utility)

The **value** of a policy is the **expected** utility.

Value: 12

Discounting



Definition: utility

Path: $s_0, a_1 r_1 s_1, a_2 r_2 s_2, \dots$ (action, reward, new state).

The utility with discount γ is:

$$u_1 = r_1 + \gamma r_2 + \gamma^2 r_3 + \gamma^3 r_4 + \dots$$

Discount $\gamma = 1$ (save for the future):

$$[\text{stay}, \text{stay}, \text{stay}, \text{stay}]: 4 + 4 + 4 + 4 = 16$$

Discount $\gamma = 0$ (live in the moment):

$$[\text{stay}, \text{stay}, \text{stay}, \text{stay}]: 4 + 0 \cdot (4 + \dots) = 4$$

Discount $\gamma = 0.5$ (balanced life):

$$[\text{stay}, \text{stay}, \text{stay}, \text{stay}]: 4 + 1/2 \cdot 4 + 1/4 \cdot 4 + 1/8 \cdot 4 = 7.5$$

Policy evaluation



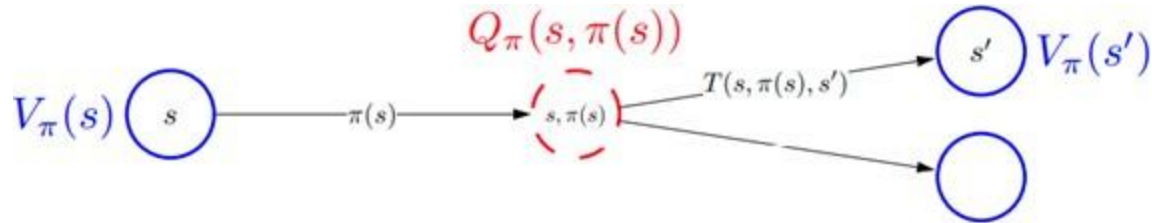
Definition: value of a policy

Let $V_{\pi}(s)$ be the expected utility received by following policy π from state s



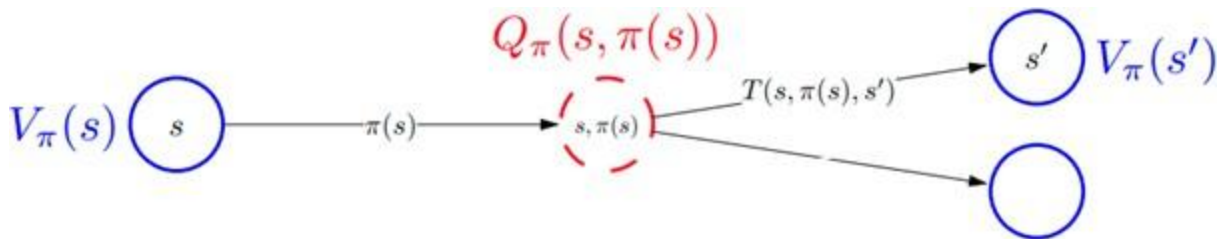
Definition: Q-value of a policy

Let $Q_{\pi}(s, a)$ be the expected utility of taking action a from state s , and then following policy π .



Policy evaluation

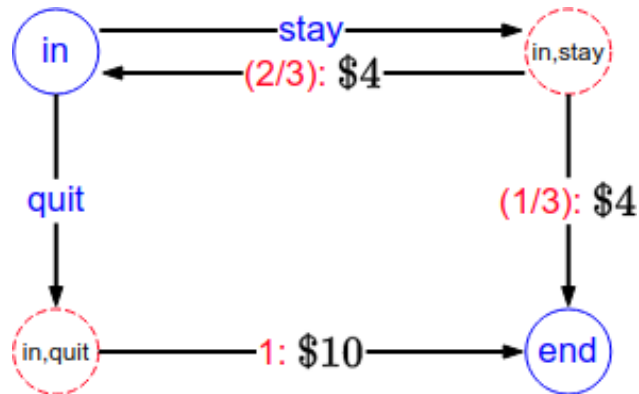
Plan: define recurrences relating value and Q-value



$$V_\pi(s) = \begin{cases} 0 & \text{if IsEnd}(s) \\ Q_\pi(s, \pi(s)) & \text{otherwise} \end{cases}$$

$$Q_\pi(s, a) = \sum_{s'} T(s'|s, a) [\text{Reward}(s, a, s') + \gamma V_\pi(s')]$$

Dice Game



(assume $\gamma = 1$)

$$V_{\pi}(s) =$$



$$Q_{\pi}(s, \pi(s))$$

if IsEnd(s)

otherwise

Let π be the "stay policy: $\pi(\text{in}) = \text{stay}$.

$$V_{\pi}(\text{end}) = 0$$

$$Q_{\pi}(s, a) = \sum_{s'} T(s'|s, a) [\text{Reward}(s, a, s') + \gamma V_{\pi}(s')]$$

$$V_{\pi}(\text{in}) = \frac{1}{3} (4 + V_{\pi}(\text{end})) + \frac{2}{3} (4 + V_{\pi}(\text{in}))$$

In this case, can solve in closed form: $V_{\pi}(\text{in}) = 12$

Policy evaluation



Key idea: iterative algorithm

Start with arbitrary policy values and repeatedly apply recurrences to converge to true values.



Algorithm: policy evaluation

Initialize $V_{\pi}^0(s) \leftarrow 0$ for all states s .

For iteration $t = 1, \dots, t_{PE}$:

For each state s :

$$V_{\pi}^{(t)}(s) \leftarrow \sum_{s'} T(s'|s, \pi(s)) [\underbrace{\text{Reward}(s, \pi(s), s') + \gamma V_{\pi}^{(t-1)}(s')}_{Q^{(t-1)}(s, \pi(s))}]$$

Policy evaluation implementation

How many iterations (t_{PE})? Repeat until values don't change much:

$$\max_{s \in \text{States}} |V_{\pi}^{(t)}(s) - V_{\pi}^{(t-1)}(s)| \leq \epsilon$$

Don't store $V_{\pi}^{(t)}$ for each iteration t , need only last two:

$$V_{\pi}^{(t)} \text{ and } V_{\pi}^{(t-1)}$$

Policy evaluation on dice game

Let π be the "stay" policy: $\pi(\text{in}) = \text{stay}$.

$$V_{\pi}^{(t)}(\text{end}) = 0$$

$$V_{\pi}^{(t)}(\text{in}) = \frac{1}{3}(4 + V_{\pi}^{(t-1)}(\text{end})) + \frac{2}{3}(4 + V_{\pi}^{(t-1)}(\text{in}))$$

s	end	in
$V_{\pi}^{(t)}$	0.00	12.00

(t = 100 iterations)

Converges to $V_{\pi}(\text{in}) = 12$.

The values of x_1 to x_{100} based on the update rule $x_{t+1} = 4 + \frac{2}{3}x_t$, starting with $x_1 = 0$, are shown below:

makefile

Copy

```
x_1 = 0
x_2 = 4.0
x_3 = 6.666666666666666
x_4 = 8.444444444444443
x_5 = 9.629629629629628
...
x_99 = 11.999999999999996
x_100 = 11.999999999999996
```

As you can see, the values quickly converge towards approximately 12, which is the stable value for this recurrence relation. [\[?\]](#)

Summary:



- **MDP:** graph with states, chance nodes, transition probabilities, rewards
- **Policy:** mapping from state to action (solution to MDP)
- **Value of policy:** expected utility over random paths
- **Policy evaluation:** iterative algorithm to compute value of policy

Outline



- Markov decision process
- Policy evaluation
- **Value iteration**

Optimal value and policy

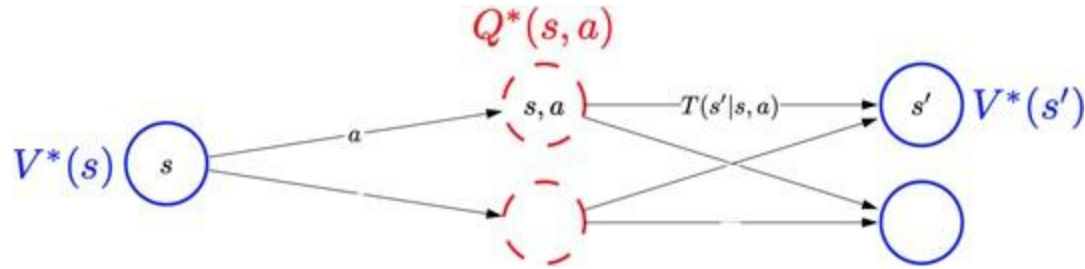
Goal: try to get directly at maximum expected utility



Definition: optimal value

The **optimal value** $V^*(s)$ is the maximum value attained by any policy.

Optimal values and Q-values



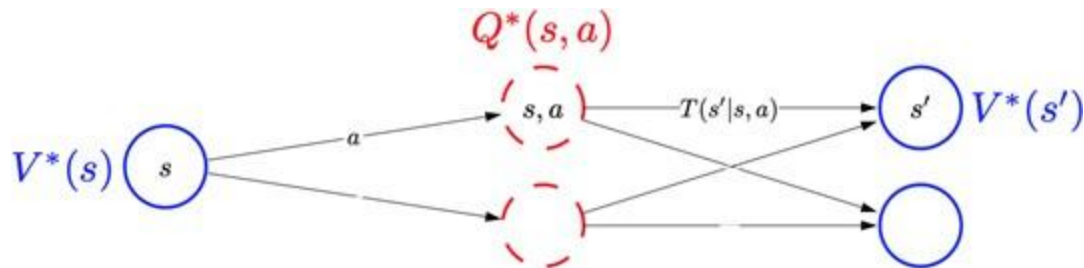
Optimal value if take action a in state s :

$$Q^*(s, a) = \sum_{s'} T(s'|s, a) [\text{Reward}(s, a, s') + \gamma V^*(s')]$$

Optimal value from state s :

$$V^*(s) = \begin{cases} 0 & \text{if IsEnd}(s) \\ \max_{a \in \text{Actions}(s)} Q^*(s, a) & \text{otherwise} \end{cases}$$

Optimal policies



Given Q^* , read off the optimal policy:

$$\pi^*(s) = \arg \max_{a \in \text{Actions}(s)} Q^*(s, a)$$

Value Iteration



Algorithm: value iteration [Bellman, 1957]

Initialize $V^{*(0)}(s) \leftarrow 0$ for all states s .

For iteration $t = 1, \dots, t_{VI}$:

For each state s :

$$V^{*(t)}(s, a) \leftarrow \max_{a \in \text{Actions}(s)} \underbrace{\sum_{s'} T(s'|s, a) [\text{Reward}(s, a, s') + \gamma V^{*(t-1)}(s')]}_{Q^{*(t-1)}(s, a)}$$



Compare them

$$V_{\pi}^{(t)}(s) \leftarrow \underbrace{\sum_{s'} T(s'|s, \pi(s)) [\text{Reward}(s, \pi(s), s') + \gamma V_{\pi}^{(t-1)}(s')]}_{Q^{(t-1)}(s, \pi(s))}$$

Value iteration: dice game

s	end	in	
$V^*(t)$	0.00	12.00	($t = 100$ iterations)
$\pi^*(s)$	-	stay	

The values of x_1 to x_{100} based on the update rule $x_{t+1} = \max(10, 4 + \frac{2}{3}x_t)$, starting with $x_1 = 0$, are as follows:

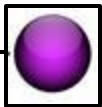
makefile

Copy

```
x_1 = 0
x_2 = 10
x_3 = 10.666666666666666
x_4 = 11.111111111111111
x_5 = 11.407407407407407
x_6 = 11.604938271604937
x_7 = 11.73662551440329
...
x_100 = 11.999999999999996
```

As you can see, the values converge to approximately 12 after the first few iterations and remain constant thereafter, with x_t stabilizing at **11.999999999999996** starting from x_2 . [↗](#)

Convergence



Theorem: convergence

Suppose either

- discount $\gamma < 1$, or
- MDP graph is acyclic.

Then value iteration converges to the correct answer.

Summary of algorithms

- **Policy evaluation:** $(\text{MDP}, \pi) \rightarrow V_\pi$
- **Value iteration:** $(\text{MDP}) \rightarrow (V^*, \pi^*)$

Algorithms:

- Policy evaluation computes policy value
- Value iteration computes optimal value

Recipe:

- Write down recurrence (e.g. $V_\pi(s) = \dots V_\pi(s') \dots$)
- Turn into iterative algorithm (replace mathematical equality with assignment operator)

Summary:



- **Markov decision processes (MDPs)** cope with uncertainty
- Solutions are **policies** rather than paths
- **Policy evaluation** computes policy value (expected utility)
- **Value iteration** computes optimal value (maximum expected utility) and optimal policy
- **Main technique:** write recurrences $\rightarrow$ algorithm
- **Next time:** reinforcement learning — when we don't know rewards, transition probabilities

U

Thanks

O



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

W